40-Day MVP Build Plan — AI College Copilot

Tracking-engine-first · solo build with Claude Code (backend) + Cursor (frontend)

> **Goal at day 40:** a deployed, working app where a faculty member logs in, sees their at-risk students with human-readable reasons, and logs an intervention — and a principal sees an institution-level view — running on one real (or realistic) college's data. Natural-language query is a **stretch goal** for the final week, not a commitment.

> **The golden rule of this plan:** ship the deploy pipeline on day 1, not day 39. Build the data foundation before any feature. Timebox the NL layer hard. If you fall behind, cut NL — never cut the tracking engine or the ingestion.

Locked stack (decide once, do not re-litigate)

Layer	Choice	Why
Backend	**Python + FastAPI**, SQLAlchemy + Alembic	Claude Code is strong here; aligns with later ML; auto OpenAPI for a typed frontend client
DB	**PostgreSQL** (`tenant_id` column on every table; app-level scoping for MVP, RLS later)	Don't build full RLS in 40 days — just never query without a `tenant_id` filter
Ingestion	CSV/Excel via **pandas**	The path that works for 100% of colleges regardless of their ERP
Frontend	**React + Vite + TypeScript + Tailwind + shadcn/ui + Recharts**	Cursor iterates fast here; charts for dashboards
Auth	Simple **JWT**, `users` table with `role` + `tenant_id`	No SSO in v1
LLM (stretch)	Anthropic API	Only for the scoped NL layer in Phase 4
Deploy	Backend+DB on **Railway/Render/Fly**; frontend on **Vercel/Netlify**	Cheap, fast, set up day 1

Phase 0 — Foundation & "deploy on day 1" (Days 1–6)

- Monorepo: `/backend`, `/frontend`, a root `PLAN.md` (this file) so the AI tools always

have project context.

- FastAPI skeleton + Postgres + Alembic migrations.
- Auth: register/login, JWT, `users(role, tenant_id)`.
- **Canonical schema** (every table has `tenant_id`, `source_system`, `source_id`): `students`, `courses`, `enrollment`, `attendance`, `internal_marks`, `fees`.
- Deploy the empty app end-to-end (frontend ↔ backend ↔ DB) so deployment is never a surprise.
- **Done when:** you can log in to a deployed app and the schema is migrated.

Phase 1 — Data ingestion (Days 7–13)

- CSV/Excel upload endpoint + a **column-mapping UI** (map their messy headers → canonical fields). This is always harder than it looks — don't skimp.
- Validation + provenance on every row.
- **Entity resolution v0:** match/dedupe students by roll-no, then name+dob fallback; assign one canonical `student_id`.
- Load one real or realistic-synthetic college's attendance + marks + fees.
- **Done when:** upload spreadsheets → unified data in DB → a Student 360 fetch returns one student's full picture.

Phase 2 — Student Success (tracking) engine (Days 14–22) ← the core IP

- **Rules engine** (write/review this yourself — it's the product, and wrong flags lose trust):
  - attendance risk: `< 75%` (configurable per tenant) or declining slope
  - academic risk: internal-marks trend vs the student's own baseline; failing-internal pattern
  - composite flag + tier (Low / Watch / High)
- Output a **risk score + plain-language reasons**; store in `risk_flags`.
- Recompute on new data (triggered or scheduled).
- **Write unit tests for the rules** — deterministic logic is easy to test and protects against regressions when the AI tools refactor.
- **Done when:** a ranked at-risk list with reasons exists, scoped per cohort/section.

Phase 3 — UI: dashboards & at-risk views (Days 23–30)

- **Student 360** page.
- **Faculty cohort risk board** (their sections only — enforce scoping).
- **Principal dashboard:** KPI tiles + at-risk count + attendance/result trends (Recharts).

- **Intervention logging** (assign → act → log outcome) — this closes the loop and creates your future ML training data.
- Alerts: email (Resend/SES); SMS/WhatsApp can be a stub for the demo.
- **Done when:** faculty sees at-risk students + reasons and logs an intervention; principal sees the institution view.

Phase 4 — Scoped NL query layer (Days 31–36) ← STRETCH, cut if behind

- **Not** open NL→SQL. Build a small **semantic layer**: 5–10 predefined metrics + allowed filters/dimensions.
- LLM maps the question → `{metric, filters, grain}` only; **deterministic code** builds the read-only query; `tenant_id` injected **server-side**; grounded answer; **"I can't answer that from your data"** fallback.
- Support a handful of question shapes well rather than everything badly.
- **Done when:** "students below 75% attendance in CSE 3rd year" and ~5 supported shapes return correct, grounded answers — and an unsupported question fails safe.

Phase 5 — Harden, polish, demo, deploy (Days 37–40)

- Tenant-isolation pass (every query filtered; try to break it).
- Seed a clean demo dataset; fix the top bugs; tighten the two screens that matter for the demo.
- Record a 5-minute demo flow; write the one-page pilot pitch (hours saved, at-risk caught).
- Final production deploy.
- **Done when:** a deployed app + a repeatable demo + one real/realistic dataset.

Scope discipline — what NOT to build in these 40 days

ML risk models · accreditation/SSR drafting · closed-system connectors (TCS iON, MasterSoft) · full RLS · SSO/MFA · native mobile · multi-channel notification depth · all five copilots (build Faculty + Principal only). All of these are post-MVP. Every one you add now puts the date at risk.

Using Claude Code + Cursor without creating a mess

- **Backend → Claude Code; frontend → Cursor.** For backend bits Cursor touches, keep them small and reviewed.
- **One source of truth for the schema** (your SQLAlchemy models / a `schema.sql`). Point both tools at it so frontend and backend never drift.

- **Lock the API contract early.** FastAPI emits OpenAPI — generate a typed TS client from it so Cursor builds against real types, not guesses.
- **Drive the agents with small, scoped tasks and review every diff.** The classic failure is letting an agent generate a sprawling tangle you can't debug by day 35. Keep modules small; commit often; one feature per branch.
- **Own the rules engine.** It's small, it's the core IP, and correctness is non-negotiable — write it deliberately and have the AI write tests for it, not the other way around.
- **Seed realistic data on day ~10** so you're never building UI against an empty database.
- **Keep `PLAN.md` updated** in the repo — it's the context that keeps both tools aligned with where you actually are.

If you fall behind (you probably will somewhere)

Cut in this order: (1) NL query layer → ship tracking-only; (2) SMS/WhatsApp alerts → email only; (3) Principal dashboard polish → keep the Faculty risk board, which is the demo. **Never cut:** ingestion, the canonical model, or the rules engine with its reasons. That trio *is* the product.